TIA — Enterprise Change Impact Intelligence Architecture

Product: TIA — Technical Impact Analysis
Companion: PRD.md v2.0
Status: Proposed architecture
Updated: 2026-10-28

Design principles

The architecture is **evidence-first**, **incremental**, **cross-repository**, and **version-aware**. TIA sits above existing engineering tools, correlates their data, and calculates the impact of a proposed change without replacing GitHub, CI/CD, static analysis, or observability.

Locked architectural decision

> Build one canonical, version-aware evidence graph with a **deterministic impact engine** above it. Connect existing tools as evidence providers. Keep **release policy** separate from **impact discovery**, and keep **report generation** separate from both.

That separation lets TIA evolve from a repository scanner into enterprise change intelligence without rebuilding the platform each time a new evidence source is added.

Layer separation (non-negotiable)

Layer	Responsibility	Must not
Source ingestion	Fetch authorised revisions & artifacts	Invent relationships
Evidence extraction	Emit normalized entities + provenance	Write opaque graph edges
Graph construction	Identity resolution, typed edges, stale invalidation	Run release policy
Impact computation	Base/head compare, traverse, classify findings	Invent missing consumers
Release-policy evaluation	Pass / fail / not_evaluated / requires_review	Auto-pass on missing evidence
Reporting	Immutable analysis snapshot → CLI / UI / PR / API	Mutate the graph

An AI model may optionally **summarise** an existing report. It must **never** invent dependencies or write confirmed edges.

---

1. High-level architecture

```
flowchart TB
  subgraph sources["1 · Engineering systems"]
    GH["GitHub Enterprise"]
    SCH["API / event schemas"]
    CI["CI/CD"]
    COV["Coverage"]
    OTEL["OpenTelemetry"]
    DEP["Deployments"]
    INC["Incidents"]
    OWN["Ownership"]
  end

  subgraph ingest["2 · Ingestion and evidence extraction"]
    RS["Repository scanner<br/>diffs · symbols · calls · endpoints · tests"]
    CS["Contract scanner<br/>API changes · schemas · compatibility"]
    CDA["CI / deployment adapters<br/>builds · test results · deployed versions"]
    ROA["Runtime / ops adapters<br/>observed calls · environments · incidents"]
  end

  subgraph norm["3 · Evidence normalization and identity resolution"]
    ID["Canonical IDs · service/repo mapping<br/>relationship typing · provenance<br/>confidence ·
  end

  subgraph graph["4 · Technical dependency graph"]
    CG["Code graph<br/>repo · commit · function · endpoint"]
    CTG["Contract graph<br/>API · event · schema · consumer"]
    VG["Verification graph<br/>test · coverage · CI result"]
    RG["Runtime graph<br/>service · artifact · deploy · trace · incident"]
  end

  subgraph engine["5 · Change-impact engine"]
    E1["1 Compare base vs proposed"]
    E2["2 Resolve changed symbols / APIs / schemas"]
    E3["3 Traverse downstream dependencies"]
    E4["4 Correlate tests · runtime · deployments"]
    E5["5 Classify findings · expose unknowns"]
  end

  subgraph out["6 · Policy and reporting"]
    IR["Impact report<br/>affected services · evidence paths · severity · unknowns"]
    PE["Release evaluation<br/>pass · fail · not evaluated · requires review"]
    SURF["CLI · Web UI · GitHub PR check · API"]
  end

  sources -->|"read-only / least privilege"| ingest
  ingest --> norm
  norm --> graph
  graph --> engine
  engine --> out
  PE --> SURF
  IR --> SURF
```

Normal flow is top → bottom. The design must also support **incremental updates**, **evidence invalidation**, **re-analysis**, and **historical queries**.

---

2. Graph model — what connects to what

The graph represents relationships needed for **change-impact questions**, not every extractable AST node.

2.1 Example: payment API changes

```
flowchart LR
  CH["Changed function / API contract<br/>Repo A · proposed commit"]
  API["Payment API / event schema<br/>contract version comparison"]
  REC["Reconciliation service<br/>Repo B"]
  FRD["Fraud-monitoring service<br/>Repo C"]
  TST["Contract test<br/>CI result · revision"]
  RUN["Runtime observation<br/>environment · time window"]

  CH -->|"EXPOSES / CHANGES"| API
  API -->|"CONSUMED BY<br/>contract-confirmed"| REC
  REC --> TST
  API -->|"CONSUMED BY<br/>runtime-observed"| FRD
  FRD --> RUN
```

Impact engine output (illustrative):

Finding	Evidence	Next check
Reconciliation	Contract relationship confirmed; relevant test identified	Run contract tests
Fraud monitoring	Runtime relationship observed; compatibility still needs verification	Verify against deployed consumer version
External consumers	Unknown if consumer inventory incomplete	Connect missing repos / registries

2.2 Entity and relationship groups

Graph group	Important entities	Example relationships
Code	Repository, commit, file, function	`CONTAINS`, `CALLS`, `CHANGED_IN`
Interfaces	API endpoint, event, schema	`EXPOSES`, `PRODUCES`, `CONSUMES`, `USES_SCHEMA`
Data	Data entity, table, migration	`READS`, `WRITES`, `MIGRATES`
Testing	Test case, test run, coverage record	`TESTS`, `EXECUTED_IN`, `COVERS`
Runtime	Service, build artifact, deployment, environment	`BUILT_FROM`, `DEPLOYED_AS`, `OBSERVED_CALL_TO`
Operations	Incident, rollback, failure signature	`ASSOCIATED_WITH`, `OCCURRED_DURING`

Graph group	Important entities	Example relationships
Governance	Owner, criticality, policy	`OWNED_BY`, `GOVERNED_BY`

Every relationship needs **evidence** and a **validity scope** (revision and/or time window and/or environment). A consumer found at commit A must not be assumed for commit B or every production environment.

2.3 Logical ER sketch (impact-relevant)

```
erDiagram
  REPOSITORY ||--o{ COMMIT : contains
  COMMIT ||--o{ FILE : contains
  FILE ||--o{ FUNCTION : declares
  FUNCTION ||--o{ FUNCTION : CALLS
  FUNCTION ||--o{ API_ENDPOINT : EXPOSES
  API_ENDPOINT ||--o{ SCHEMA : USES_SCHEMA
  SERVICE ||--o{ API_ENDPOINT : owns
  SERVICE ||--o{ SERVICE : CONSUMES_API
  SERVICE ||--o{ EVENT : PRODUCES_OR_CONSUMES
  FUNCTION ||--o{ TEST_CASE : TESTS
  TEST_CASE ||--o{ TEST_RUN : EXECUTED_IN
  SERVICE ||--o{ DEPLOYMENT : DEPLOYED_AS
  DEPLOYMENT }o--|| BUILD_ARTIFACT : uses
  BUILD_ARTIFACT }o--|| COMMIT : BUILT_FROM
  SERVICE ||--o{ OBSERVED_CALL : OBSERVED_CALL_TO
  SERVICE ||--o{ INCIDENT : ASSOCIATED_WITH
  SERVICE }o--|| OWNER : OWNED_BY
  SERVICE }o--o| POLICY : GOVERNED_BY
```

---

3. Backend services and responsibilities

Logically separate; **not** all required as separate microservices in MVP.

```
flowchart LR
  subgraph clients["Clients"]
    CLI["CLI"]
    UI["Web UI"]
    PR["GitHub App / checks"]
    MCP["Optional MCP read"]
  end

  RPT["F · Report and API service"]
  POL["E · Policy engine"]
  IMP["D · Impact-analysis engine"]
  GRW["C · Identity resolution and graph writer"]
  EXT["B · Extraction workers"]
  CON["A · Connector service"]

  clients --> RPT
  RPT --> POL
  RPT --> IMP
  IMP --> GRW
  POL --> IMP
  EXT --> GRW
  CON --> EXT
```

Service	Responsibility
A. Connector service	Retrieve authorised revisions, contracts, CI results, runtime evidence, deployments, incidents. Pagination, retries, permissions, incremental updates.
B. Extraction workers	Language parsers, framework adapters, schema diffing, test-report parsers. Emit **normalized entities + evidence records** — not arbitrary graph writes.
C. Identity resolution & graph writer	Cross-repo identity, dedupe, typed relationships, provenance, stale invalidation.
D. Impact-analysis engine	Changed components, downstream traversal, contract diffs, relevant tests, deployment correlation, structured findings.
E. Policy engine	Customer release rules vs findings. Missing evidence → `NOT_EVALUATED` / `REQUIRES_REVIEW`, never automatic pass.
F. Report and API service	Impact reports, evidence paths, status, history, policy outcomes for UI / CLI / GitHub.

---

4. Storage architecture

Separate **logical** stores for different access patterns. You do not need a different database product for every box.

```
flowchart TB
  EXT["Extraction / connectors"] --> META["Relational DB<br/>repos · scans · jobs · revisions<br/>a
  EXT --> EVD["Evidence / artifact store<br/>scan manifests · contract snapshots<br/>test reports ·
  EXT --> GRP["Graph store<br/>entities · typed edges<br/>evidence links · temporal validity"]
  EXT -.-> TEL["Optional telemetry store / connector<br/>runtime observations · trace refs"]

  IMP["Impact engine"] --> GRP
  IMP --> META
  IMP --> EVD
  IMP -.-> TEL
  RPT["Reports / API"] --> META
  RPT --> EVD
```

Store	Contents	Purpose
Graph store	Entities, typed edges, evidence links, temporal relationships	Dependency traversal and impact discovery
Relational DB	Repositories, scans, jobs, revisions, analysis runs, policies, status	Transactional metadata and workflow state
Evidence/artifact store	Scan manifests, contract snapshots, test reports, selected source evidence	Reproducibility and audit
Optional telemetry	Runtime observations and trace references	Time-windowed execution evidence

Prototype: SQLite + local evidence directory.
Enterprise: PostgreSQL for workflow/metadata; graph store (RDF or property graph) chosen after benchmarking traversal. Avoid locking architecture to proprietary graph features.

Important: Do **not** copy entire repositories, every trace, and every test log into the graph. Store compact relationships and **stable references** to detailed evidence.

---

5. End-to-end pull-request flow

```
sequenceDiagram
  participant GH as GitHub
  participant API as Report / API
  participant CON as Connectors
  participant EXT as Extractors
  participant GR as Graph writer
  participant IMP as Impact engine
  participant POL as Policy engine
  participant ENG as Engineer

  GH->>API: PR opened / updated (webhook)
  API->>API: Create analysis job
  API->>CON: Resolve base and head revisions
  CON->>EXT: Extract changed files and symbols
  EXT->>GR: Update source evidence
  GR->>GR: Resolve entities and relationships
  IMP->>GR: Build scoped impact subgraph
  IMP->>IMP: Traverse downstream dependencies
  IMP->>IMP: Compare contracts and schemas
  IMP->>CON: Correlate tests, runtime, deployments
  IMP->>API: Generate evidence-backed findings
  POL->>API: Evaluate release policies
  API->>API: Persist immutable analysis snapshot
  API->>GH: Publish PR report and status
  ENG->>GH: Review findings / unknowns
```

Processing rules

1. A new commit invalidates only **affected** evidence — do not rebuild the entire enterprise graph unnecessarily.
2. **Identity resolution before** impact traversal — otherwise two references to the same service look unrelated.
3. Traversal follows **typed** relationships — test association ≠ runtime observation ≠ direct call.
4. The analysis result is a **snapshot**. If the graph changes later, preserve the original report; create a new analysis when required.
5. The GitHub check reflects analysis status. An **incomplete** analysis must not be reported as a clean impact assessment.

---

6. Recommended technology stack

Practical starting point — not a mandate to adopt everything immediately.

Component	Starting point	Reason
GitHub integration	GitHub App + webhooks	Controlled permissions and PR lifecycle
API / backend	Python + FastAPI	Orchestration and integration APIs
Python analysis	Tree-sitter or `ast` + framework extractors	Symbols and relationships
Java analysis	Tree-sitter / Java-aware adapter	When Java enters pilot scope

Component	Starting point	Reason
API contracts	OpenAPI parser + schema diff	Endpoints and compatibility
Event schemas	Format-specific adapters	Producer/consumer impact
Workflow metadata	PostgreSQL (enterprise)	Durable jobs, policies, concurrency
Graph storage	RDF or property graph (benchmark)	Typed traversal + evidence
Evidence storage	S3-compatible / customer-managed	Artifacts and audit
Background jobs	Postgres-backed queue first	Retryable incremental scans
UI	React	Reports and evidence-path explorer
Deployment	Containers, customer-controlled	Private / restricted environments

Do not introduce Kafka, Kubernetes sprawl, multiple graph DBs, vector search, or an LLM agent framework "because enterprise." Add only when throughput, deployment, or retrieval requirements justify them.

---

7. API boundaries

CLI, UI, GitHub, and future clients share the same analysis engine.

Endpoint	Responsibility
`POST /api/v1/repositories`	Register an authorised repository
`POST /api/v1/scans`	Request a repository or revision scan
`GET /api/v1/scans/{scan_id}`	Scan status and coverage
`POST /api/v1/impact-analyses`	Start impact analysis for base/head
`GET /api/v1/impact-analyses/{analysis_id}`	Report and findings
`GET /api/v1/impact-analyses/{analysis_id}/paths`	Evidence paths for findings
`GET /api/v1/components/{component_id}/dependencies`	Scoped dependency information
`GET /api/v1/policies`	List configured release policies
`POST /api/v1/policy-evaluations`	Evaluate a report against policies

All endpoints enforce authorisation and repository scope. Scans and impact analyses are **asynchronous jobs**, not blocking HTTP requests.

---

8. Deployment topology (bank)

```
flowchart TB
  subgraph bank["Bank engineering environment"]
    B1["GitHub Enterprise"]
    B2["CI/CD"]
    B3["Schema registry"]
    B4["Observability"]
  end

  subgraph tia["Private TIA deployment"]
    API["API and integration<br/>auth · webhooks · access control"]
    WRK["Worker pool<br/>scanning · parsing · graph updates"]
    DATA["Data layer<br/>graph · PostgreSQL · evidence store"]
    AUD["Audit and monitoring<br/>access logs · job health · retention"]
  end

  subgraph back["Results returned to engineering workflow"]
    R1["Impact report"]
    R2["Evidence paths"]
    R3["Required checks"]
    R4["Policy result"]
  end

  bank -->|"least privilege"| tia
  tia --> back
```

For restricted environments, repository content and evidence remain inside the customer-approved boundary. Outbound connections, telemetry retention, and any optional AI summarisation must be explicitly controlled.

---

9. Build order

```
flowchart LR
  P1["1 Foundation<br/>GitHub · base/head<br/>Python extract · scan tracking · provenance"]
  P2["2 First useful graph<br/>cross-repo APIs · typed traversal<br/>evidence-backed JSON report"]
  P3["3 Verification<br/>test links · contract compatibility<br/>CI results"]
  P4["4 Production context<br/>runtime · deploy versions<br/>ownership"]
  P5["5 Enterprise control<br/>policy · GitHub checks<br/>audit · access · private deploy"]

  P1 --> P2 --> P3 --> P4 --> P5
```

Step	Outcome
1 Foundation	GitHub ingestion, base/head comparison, Python extraction, scan tracking, provenance
2 First useful graph	Cross-repository API relationships, typed dependency traversal, evidence-backed JSON report
3 Verification	Test associations, contract compatibility, CI result ingestion
4 Production context	Runtime observations, deployment versions, service ownership

---

Step	Outcome
5 Enterprise control	Policy evaluation, GitHub checks, audit, access control, private deployment hardening

---

10. Trust boundary (impact vs AI)

```
flowchart TB
  EVID["Evidence providers<br/>GitHub · schemas · CI · OTEL · deploy · incidents"]
  DET["Deterministic path<br/>extract → resolve → graph → impact engine → policy"]
  RPT["Immutable Impact Report"]
  LLM["Optional LLM<br/>summarise report only"]
  AGENT["Coding assistant / MCP<br/>read report by analysis id"]

  EVID --> DET --> RPT
  RPT --> LLM
  RPT --> AGENT

  LLM -.->|"forbidden: invent edges / fill unknowns"| DET
  AGENT -.->|"forbidden: write confirmed relationships"| DET
```

Document history

Version	Date	Notes
1.0	2026-10-28	Initial architecture from enterprise TIA design; Mermaid diagrams for HL view, graph example, services, storage, PR flow, deploy, build order, trust boundary

---

End of architecture